

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Programming Integration Project (Học kỳ 252)

Assignment

Boolean Network Analyzer – Phân tích, mô phỏng và trực quan hoá mạng Boolean

Instructor(s): Trịnh Văn Giang

Student(s): Trần Phan Đăng Khôi 2352626

HO CHI MINH CITY, MAY 2026

Tóm tắt

Đề tài (Chủ đề 6): BOOLEAN NETWORK ANALYZER – một phần mềm phân tích, mô phỏng và trực quan hoá Mạng Boolean.

Mạng Boolean (Boolean Network – BN) là một mô hình toán học rời rạc được Stuart Kauffman đề xuất năm 1969 nhằm mô phỏng các *mạng điều hoà gene* (gene regulatory network). Mỗi node trong mạng nhận một trong hai giá trị $\{0, 1\}$ và được cập nhật theo một hàm Boolean phụ thuộc vào trạng thái của các node lân cận. Bất chấp sự đơn giản về biểu diễn, BN đủ mạnh để mô tả *các trạng thái ổn định* (steady states), *các vòng dao động* (cyclic attractor) và *các hành vi phức tạp* (complex attractor) của những hệ thống sinh học và logic công nghiệp thực tế. Trên thực tế, các bộ công cụ chuẩn quốc tế như GINSIM, PYBOOLNET, BOOLNET (R), CELLNETANALYZER đã trở thành chuẩn de facto cho nghiên cứu dynamics của BN.

Báo cáo này trình bày quá trình *khảo sát lý thuyết, thiết kế kiến trúc, hiện thực và đánh giá* một phần mềm BOOLEAN NETWORK ANALYZER hoàn chỉnh dưới dạng ứng dụng web tương tác. Phần mềm hỗ trợ năm khối chức năng bắt buộc: (i) **Parser** đọc file `.bnet` chuẩn (tương thích PyBoolNet) gồm các luật cập nhật dạng `target, expression`; (ii) **Influence Graph Builder** dựng đồ thị ảnh hưởng có dấu (+/-/?), với việc suy luận dấu dựa trên kiểm tra tính đơn điệu (monotonicity) của hàm Boolean; (iii) **State Transition Graph (STG) Engine** duyệt toàn bộ 2^n trạng thái dưới hai sơ đồ cập nhật *Synchronous* và *Asynchronous*; (iv) **Attractor Finder** áp dụng phân rã thành phần liên thông mạnh (SCC – Tarjan) trên đồ thị condensation để xác định mọi attractor và phân loại thành *fixed-point/ cyclic /complex*; (v) **Visualizer** hiển thị tương tác mọi đồ thị trên giao diện Web (Landing Page), có khả năng export PNG cho báo cáo.

Bốn đóng góp chính của đề tài: (1) **Kiến trúc module rõ ràng** (Parser → Network → STG → Attractors → Visualizer) tách biệt I/O và logic; (2) **Bộ kiểm thử tự động (pytest)** bao phủ Parser, Network, STG, Attractors và các sample chuẩn; (3) **Đối chiếu kết quả với công cụ chuẩn PyBoolNet** trên năm mạng mẫu (*toy switch, bistable toggle, repressilator, fission yeast, mammalian cell cycle*) – kết quả tính toán attractor trùng khớp 100%; (4) **Giao diện Landing Page thân thiện** cho người dùng không lập trình, tích hợp upload file, lựa chọn sample, chuyển đổi sơ đồ cập nhật, mô phỏng quỹ đạo và tải về hình.

Hệ thống đã được kiểm thử với năm mạng từ 2 đến 10 nodes; cả năm trường hợp đều cho kết quả attractor đúng theo tài liệu gốc (Davidich & Bornholdt 2008, Faure *et al.* 2006, Gardner *et al.* 2000). Đánh giá hiệu năng cho thấy phần mềm chạy mượt với mạng



≤ 16 nodes (tương ứng $2^{16} = 65\,536$ trạng thái); vượt ngưỡng này, thời gian sinh STG đầy đủ tăng theo cấp số mũ và buộc người dùng chuyển sang chế độ *reachable-only* (chỉ duyệt các trạng thái đạt tới được từ trạng thái khởi đầu). Toàn bộ source code, dataset mẫu và tài liệu được tổ chức theo chuẩn project layout chuyên nghiệp (`src/`, `tests/`, `samples/`, `docs/`) và công khai trên repository GitHub tại: <https://github.com/dangkhoei-dev/boolean-network-analyzer>.

Từ khoá: Boolean Network, Influence Graph, State Transition Graph, Attractor, Synchronous Update, Asynchronous Update, Strongly Connected Component, PyBoolNet, Web UI, Landing Page, Bioinformatics.



Contents

Tóm tắt	i
Danh sách hình vẽ	vi
Danh sách bảng	vii
1 Giới thiệu	1
1.1 Bối cảnh	1
1.1.1 Mạng Boolean trong khoa học hệ thống	1
1.1.2 Ứng dụng của Boolean Network	1
1.1.3 Khoảng trống công cụ và động lực	2
1.2 Mục tiêu dự án	2
1.3 Phạm vi đề tài	3
1.4 Đóng góp chính	3
1.5 Cấu trúc báo cáo	4
2 Cơ sở lý thuyết	4
2.1 Định nghĩa toán học của Boolean Network	4
2.1.1 Định nghĩa hình thức	4
2.1.2 Biểu diễn rule trong định dạng .bnet	5
2.1.3 Ví dụ minh họa: mạng <i>toy switch</i> 3 node	5
2.2 Đồ thị ảnh hưởng (Influence Graph)	6
2.2.1 Định nghĩa	6
2.2.2 Suy luận dấu cạnh	7
2.2.3 Ví dụ minh họa	7
2.2.4 Vai trò của Influence Graph	8
2.3 Sơ đồ cập nhật: Synchronous vs Asynchronous	8
2.3.1 Sơ đồ Synchronous	9
2.3.2 Sơ đồ Asynchronous	9
2.3.3 Ví dụ so sánh: bistable toggle 2 node	10
2.4 State Transition Graph (STG)	11
2.4.1 Định nghĩa	11
2.4.2 Kích thước và biểu diễn	11
2.4.3 Ví dụ: STG synchronous của <i>repressilator</i>	12
2.5 Attractor: Point & Cyclic & Complex	13



2.5.1	Định nghĩa	13
2.5.2	Phân loại attractor	13
2.5.3	Vùng hút (Basin of Attraction)	14
2.5.4	Ý nghĩa sinh học và kỹ thuật	14
2.6	Đối chiếu với các công cụ chuẩn	15
3	Phương pháp & Thiết kế hệ thống	15
3.1	Kiến trúc tổng thể	15
3.2	Tổ chức source code và package layout	17
3.3	Cấu trúc dữ liệu trong bộ nhớ	18
3.3.1	ParsedRule	18
3.3.2	State	18
3.3.3	Graphs: NetworkX DiGraph	19
3.4	Thuật toán cốt lõi	19
3.4.1	Thuật toán Parser	19
3.4.2	Thuật toán Evaluate (network)	19
3.4.3	Thuật toán xây Influence Graph	20
3.4.4	Thuật toán xây STG	20
3.4.5	Thuật toán tìm Attractor: SCC + Condensation	21
3.4.6	Thuật toán tính Basin of Attraction	22
3.5	Thiết kế giao diện người dùng (Landing Page & Web UI)	22
3.6	Tổng kết Methodology	24
4	Thực nghiệm & Đánh giá	24
4.1	Bộ test case mẫu	24
4.2	Kết quả từng test case	25
4.2.1	Test case 1: <code>toy_switch</code> (3 nodes)	26
4.2.2	Test case 2: <code>bistable_toggle</code> (2 nodes)	26
4.2.3	Test case 3: <code>repressilator</code> (3 nodes)	27
4.2.4	Test case 4: <code>fission_yeast</code> (9 nodes)	27
4.2.5	Test case 5: <code>mammalian_cell_cycle</code> (10 nodes)	28
4.3	Xác thực tính đúng đắn (Validation)	28
4.3.1	Đối chiếu với PyBoolNet	28
4.3.2	Đối chiếu với tính toán thủ công (manual check)	29
4.3.3	Bộ kiểm thử tự động (pytest)	30
4.4	Phân tích hiệu năng	31



4.5	Một số bug phát hiện và sửa trong quá trình develop	32
4.6	Tổng kết Thực nghiệm	32
5	Kết luận & Hướng phát triển	32
5.1	Tổng kết	32
5.2	Hạn chế	34
5.3	Hướng phát triển	34
5.4	Lời cảm ơn	35
	Tài liệu tham khảo	36



List of Figures

2.1	Influence Graph của mạng <i>toy switch</i> 3 node. Cạnh xanh: activation (+); cạnh đỏ: inhibition (-).	8
2.2	State Transition Graph (async) của bistable toggle. Hai node màu cam là hai fixed-point attractor; các trạng thái 00 và 11 là transient.	11
2.3	STG synchronous của repressilator. Các node màu cam thuộc về attractor; vòng 6 trạng thái biểu diễn dao động sinh học, vòng 2 trạng thái là artifact của sync.	13
3.1	Kiến trúc tổng thể phần mềm Boolean Network Analyzer	16
3.2	Giao diện trang chủ giới thiệu phần mềm Boolean Network Analyzer. . . .	23
3.3	Giao diện Live Demo trên trình duyệt, đang hiển thị Đồ thị ảnh hưởng (Influence Graph) của mạng tế bào Mammalian 10 nodes.	23
4.1	Tổng hợp kết quả render trực quan đồ thị STG và Influence Graph cho mạng Repressilator, Bistable Toggle và Toy Switch từ giao diện phần mềm. .	25



List of Tables

1.1	Phạm vi triển khai của đề tài Boolean Network Analyzer	3
2.1	Sync vs Async cho mạng bistable toggle	10
2.2	Tăng trưởng kích thước STG theo n	12
2.3	So sánh các công cụ phân tích Boolean Network	15
3.1	Bảng chuyển token .bnet $\rightarrow$ Python	19
4.1	Năm mạng Boolean mẫu được dùng để kiểm thử	24
4.2	Bảng chuyển trạng thái sync cho <code>toy_switch</code>	26
4.3	Đối chiếu kết quả attractor BNAnalyzer vs PyBoolNet (FP = fixed point)	29
4.4	Thời gian sinh STG và attractor trên mạng tổng hợp (đo trên Intel i5-1135G7 @ 2.4GHz, 16GB RAM)	31

1 Giới thiệu

1.1 Bối cảnh

1.1.1 Mạng Boolean trong khoa học hệ thống

Mạng Boolean (Boolean Network – BN) là một loại mô hình toán học rời rạc dùng để mô tả *động học* (dynamics) của các hệ thống mà mỗi thành phần chỉ có thể ở một trong hai trạng thái rời rạc: *bật* (1, **True**, ON, active) hoặc *tắt* (0, **False**, OFF, inactive). Mô hình này được Stuart A. Kauffman đề xuất lần đầu vào năm 1969^[2] trong bối cảnh nghiên cứu *mạng điều hoà gene* (gene regulatory network – GRN). Ý tưởng cốt lõi là: thay vì mô hình hoá chính xác nồng độ hay tốc độ phản ứng bằng các hệ phương trình vi phân thường (ODE) – vốn đòi hỏi nhiều tham số khó đo đạc – ta *trừu tượng hoá* mỗi gene thành một biến nhị phân và mỗi quan hệ điều hoà thành một biểu thức logic.

Bất chấp sự đơn giản về biểu diễn, BN đã chứng tỏ là một công cụ mạnh, có khả năng tái hiện nhiều hiện tượng định tính quan trọng quan sát được trong sinh học và kỹ thuật như: tồn tại nhiều *trạng thái ổn định* (multistability) tương ứng với các kiểu hình tế bào khác nhau, *dao động tuần hoàn* (oscillation) trong chu kỳ tế bào, *tính chống nhiễu* (robustness) trước nhiễu ngẫu nhiên, và sự tồn tại của các *vùng hút phức tạp* (complex attractor) trong các mạng cập nhật bất đồng bộ.

1.1.2 Ứng dụng của Boolean Network

Hiện nay, BN không chỉ được dùng trong sinh học hệ thống mà còn được áp dụng cho nhiều lĩnh vực khác:

- **Sinh học phân tử và y sinh:** Mô hình hoá chu kỳ tế bào (*S. pombe*, *S. cerevisiae*, mammalian cell cycle), mạng tín hiệu của tế bào (TGF- β , MAPK, T-cell receptor), mô phỏng tiến trình apoptosis và sự lây nhiễm virus^[1;8;9].
- **Thiết kế mạch logic số:** Mỗi cổng logic được trừu tượng hoá thành một node BN; phân tích STG cho phép tìm racing condition và glitch trong mạch tổ hợp/tuần tự.
- **Hệ điều khiển công nghiệp PLC:** Các quan hệ logic giữa cảm biến/cơ cấu chấp hành có thể mô hình hoá bằng BN, từ đó kiểm chứng formal cho safety property.
- **Khoa học xã hội tính toán:** BN được dùng để mô phỏng *lan truyền ý kiến* (opinion dynamics) trên mạng xã hội, mỗi cá nhân có quan điểm 0/1 phụ thuộc vào láng giềng.

1.1.3 Khoảng trống công cụ và động lực

Mặc dù đã có các bộ công cụ chuyên nghiệp như GINSIM^[3], PYBOOLNET^[4], BOOL-NET (R) và CELLNETANALYZER, các công cụ này thường có *rào cản tiếp cận cao* với người học mới: GINSim đòi hỏi cài đặt JVM và GUI khá phức tạp; PyBoolNet là thư viện Python thuần, không có giao diện trực quan; BoolNet yêu cầu môi trường R. Đối với sinh viên năm nhất/năm hai tiếp cận môn *Programming Integration Project*, việc có một *ứng dụng web nhẹ, đa nền tảng*, cho phép tải file **.bnet** lên và ngay lập tức nhìn thấy đồ thị ảnh hưởng cùng các attractor là rất hữu ích cho cả việc học và việc trình bày kết quả.

Đó chính là động lực để chúng em xây dựng BOOLEAN NETWORK ANALYZER – một phần mềm đầy đủ chức năng phân tích cơ bản của BN, giao diện web tương tác (Landing Page), mã nguồn mở Python và tổ chức theo chuẩn project chuyên nghiệp để có thể tái sử dụng cho các đề án sau này.

1.2 Mục tiêu dự án

Đề tài **Boolean Network Analyzer** hướng tới việc xây dựng một phần mềm có khả năng:

1. **Đọc và phân tích cú pháp** file **.bnet** chuẩn (tương thích PyBoolNet/BoolNet) gồm các luật cập nhật dạng **target**, **expression** với các toán tử logic **!**, **&**, **|** và dấu ngoặc.
2. **Xây dựng đồ thị ảnh hưởng** (Influence Graph) có dấu (+ activation / – inhibition / ? non-monotonic) bằng cách suy luận trực tiếp từ luật cập nhật.
3. **Dựng đồ thị chuyển trạng thái** (State Transition Graph – STG) đầy đủ trên toàn bộ 2^n trạng thái dưới *hai* sơ đồ cập nhật chuẩn: **synchronous** và **asynchronous**.
4. **Tìm attractor** (terminal SCC) bằng thuật toán SCC chuẩn (Tarjan/Kosaraju), phân loại thành *fixed-point*, *cyclic* hoặc *complex*, và tính *vùng hút* (*basin of attraction*) của mỗi attractor.
5. **Trực quan hoá** mọi đồ thị (Influence Graph, STG) với layout tự động, đánh dấu attractor bằng màu, hỗ trợ export PNG.
6. **Mô phỏng quỹ đạo** (trajectory simulation) từ trạng thái khởi đầu bất kỳ do người dùng chọn, hỗ trợ cả sync lẫn async.
7. **Cung cấp bộ sample** có sẵn (5 mạng từ kinh điển đến phức tạp) cho phép kiểm thử nhanh.

1.3 Phạm vi đề tài

Đề tài tập trung vào phần *phân tích định tính* của BN – cụ thể là cấu trúc topological (influence graph) và động học rời rạc (STG, attractor). Phạm vi cụ thể được tóm tắt trong Bảng 1.1.

Thành phần	Phạm vi triển khai
Parser .bnet	Tương thích cú pháp PyBoolNet/BoolNet; hỗ trợ toán tử <code>!, &, , ()</code> ; chấp nhận hằng <code>0, 1, True, False</code> ; bỏ qua header <code>targets, factors</code> và comment <code>#</code> .
Network Engine	Lớp <code>BooleanNetwork</code> với evaluation toàn bộ rule; tính influence graph có dấu; chuyển đổi giữa state-tuple, dict, binary-string.
STG Builder	Hỗ trợ <i>full state space</i> (2^n states) và <i>reachable-only</i> (BFS từ trạng thái khởi đầu); hai sơ đồ sync & async.
Attractor Finder	SCC decomposition (Tarjan via NetworkX); classification fixed-point / cyclic / complex; tính weak basin.
Visualizer	Render influence graph bằng <code>matplotlib</code> ; STG có highlight attractor; export PNG độ phân giải 160–180 dpi.
Web UI	Web SPA (Landing Page) trực quan: tích hợp các phân hệ hiển thị Influence Graph, STG, Attractors và Simulate.
Test & Validation	4 file <code>pytest</code> : parser, network, stg+attractors, samples. Đối chiếu thủ công với PyBoolNet.

Table 1.1: Phạm vi triển khai của đề tài Boolean Network Analyzer

1.4 Đóng góp chính

Sau khi hoàn thành, phần mềm cung cấp các đóng góp:

1. **Bộ phân tích Boolean Network mã nguồn mở:** Cài đặt lại các thuật toán nền (parser, evaluation, STG, SCC) bằng Python thuần, có thể đọc hiểu, sửa đổi và mở rộng dễ dàng.

2. **Giao diện Landing Page tương tác:** Người dùng không cần biết lập trình vẫn có thể upload file `.bnet`, nhìn thấy ngay đồ thị và attractor.
3. **Đối chiếu với công cụ chuẩn PyBoolNet:** Kết quả attractor trên 5 mạng mẫu khớp 100% với PyBoolNet – chứng minh tính đúng đắn của hiện thực.
4. **Bộ kiểm thử tự động:** 30+ test case `pytest` bao phủ parser, network evaluation, STG construction, attractor classification và smoke test trên các sample.
5. **Tổ chức project chuyên nghiệp:** Tách `src/`, `tests/`, `samples/`, `docs/`; có `requirements.txt`, `README.md`, `LICENSE`; tương thích CI/CD GitHub Actions. Mã nguồn mở tại: <https://github.com/dangkhoei-dev/boolean-network-analyzer>.

1.5 Cấu trúc báo cáo

Phần còn lại của báo cáo được tổ chức như sau. Mục 2 trình bày *cơ sở lý thuyết* về BN bao gồm định nghĩa toán học, influence graph, hai sơ đồ cập nhật và khái niệm attractor. Mục 3 mô tả *kiến trúc tổng thể và thuật toán cốt lõi* của phần mềm. Mục 4 trình bày *kết quả thực nghiệm và đánh giá đối chiếu* với PyBoolNet trên 5 mạng mẫu. Mục 5 *kết luận* và đề xuất *hướng phát triển* trong tương lai. Tài liệu tham khảo được liệt kê đầy đủ ở cuối báo cáo.

2 Cơ sở lý thuyết

Mục này trình bày các khái niệm toán học làm nền tảng cho phần mềm Boolean Network Analyzer. Trình bày dựa trên hai nguồn chính: bài tổng quan của Schwab *et al.*^[1] và tài liệu của Naldi *et al.*^[3]. Các định nghĩa được tự diễn đạt lại bằng tiếng Việt kèm ví dụ minh họa tự thiết kế nhằm thể hiện sự hiểu rõ bản chất chứ không sao chép máy móc.

2.1 Định nghĩa toán học của Boolean Network

2.1.1 Định nghĩa hình thức

Một Boolean Network với n node là một bộ ba

$$\mathcal{B} = (V, \mathcal{F}, \mathcal{X})$$

trong đó:

- $V = \{v_1, v_2, \dots, v_n\}$ là tập n node (gene, biến, thành phần).
- $\mathcal{X} = \{0, 1\}^n$ là không gian trạng thái chứa 2^n trạng thái. Một trạng thái cụ thể $x = (x_1, x_2, \dots, x_n) \in \mathcal{X}$ gán cho mỗi node một giá trị Boolean.
- $\mathcal{F} = \{f_1, f_2, \dots, f_n\}$ là tập hàm cập nhật (update functions), trong đó $f_i : \{0, 1\}^n \rightarrow \{0, 1\}$ xác định giá trị tiếp theo của node v_i dựa trên trạng thái hiện tại.

Trong thực tế, f_i thường chỉ phụ thuộc vào một tập con các node $\text{In}(v_i) \subseteq V$ – gọi là *regulator set* (tập điều hoà) của v_i . Số phần tử $|\text{In}(v_i)|$ là *in-degree* của v_i trong influence graph (xem §2.2).

2.1.2 Biểu diễn rule trong định dạng .bnet

Định dạng .bnet (chuẩn của BoolNet R-package và PyBoolNet) biểu diễn mỗi rule trên một dòng riêng theo cú pháp:

Listing 1: Cú pháp file .bnet

```
1 # Comments start with '#' and run to end of line.
2 # Header (optional, ignored by the parser):
3 targets, factors
4 # One rule per line:
5 A, !B
6 B, A & !C
7 C, B | !A
```

Các toán tử logic được hỗ trợ là ! (NOT), & (AND), | (OR) và dấu ngoặc đơn. Các hằng 0, 1, True, False cũng được nhận diện. Ưu tiên toán tử tuân theo chuẩn Python: NOT > AND > OR.

2.1.3 Ví dụ minh hoạ: mạng *toy switch* 3 node

Để minh hoạ, xét mạng 3 node *toy switch* trong Listing 2:

Listing 2: Mạng *toy switch* 3 node

```
1 targets, factors
2 A, !B
3 B, A & !C
4 C, B
```

Mỗi node có một rule:

$$f_A(x_A, x_B, x_C) = \neg x_B$$

$$f_B(x_A, x_B, x_C) = x_A \wedge \neg x_C$$

$$f_C(x_A, x_B, x_C) = x_B$$

Không gian trạng thái có $2^3 = 8$ trạng thái, đánh số bằng chuỗi nhị phân ABC: 000, 001, 010, 011, 100, 101, 110, 111. Ví dụ, từ trạng thái $x = (1, 0, 0)$ ($A = 1, B = 0, C = 0$) ta tính:

$$f_A = \neg 0 = 1$$

$$f_B = 1 \wedge \neg 0 = 1$$

$$f_C = 0$$

Nghĩa là trạng thái kế tiếp (theo sơ đồ *synchronous*) là $(1, 1, 0)$. Lặp lại bước này cho toàn bộ 8 trạng thái sẽ thu được STG đầy đủ (xem §2.4).

2.2 Đồ thị ảnh hưởng (Influence Graph)

2.2.1 Định nghĩa

Influence Graph $G_I = (V, E_I, \sigma)$ của một *Boolean Network* $\mathcal{B}$ là một đồ thị có hướng, có dấu, trong đó:

- Tập đỉnh V trùng với tập node của BN.
- Tồn tại cạnh có hướng $(u, v) \in E_I$ khi và chỉ khi $u \in \text{In}(v)$, tức u xuất hiện trong rule của v .
- $\sigma : E_I \rightarrow \{+, -, ?\}$ là hàm gán dấu cho mỗi cạnh:
 - $\sigma(u, v) = +$ (activation) nếu việc bật u luôn làm tăng (hoặc giữ nguyên) giá trị f_v .
 - $\sigma(u, v) = -$ (inhibition) nếu việc bật u luôn làm giảm (hoặc giữ nguyên) giá trị f_v .
 - $\sigma(u, v) = ?$ (non-monotonic) nếu hành vi phụ thuộc vào trạng thái của các node khác.

2.2.2 Suy luận dấu cạnh

Cho rule f_v và regulator $u \in \text{In}(v)$. Đặt $\text{In}(v) \setminus \{u\} = \{w_1, \dots, w_k\}$. Với mỗi gán giá trị $(b_1, \dots, b_k) \in \{0, 1\}^k$ cho các regulator còn lại, ta so sánh:

$$\Delta_u(b_1, \dots, b_k) = f_v(u = 1, b_1, \dots, b_k) - f_v(u = 0, b_1, \dots, b_k) \in \{-1, 0, +1\}$$

Có ba khả năng:

- Nếu $\Delta_u \geq 0$ với mọi $(b_1, \dots, b_k)$ và $\Delta_u > 0$ ít nhất một lần $\Rightarrow \sigma(u, v) = +$.
- Nếu $\Delta_u \leq 0$ với mọi $(b_1, \dots, b_k)$ và $\Delta_u < 0$ ít nhất một lần $\Rightarrow \sigma(u, v) = -$.
- Nếu vừa có $\Delta_u > 0$ vừa có $\Delta_u < 0$ ở các gán khác nhau $\Rightarrow \sigma(u, v) = ?$.

Thuật toán này có độ phức tạp $O(2^k)$ với $k = |\text{In}(v)| - 1$. Vì k thường nhỏ (≤ 5 trong các mạng sinh học thực tế), việc suy luận dấu là rất nhanh.

2.2.3 Ví dụ minh họa

Áp dụng cho mạng *toy switch*:

- Edge $B \rightarrow A$: $f_A = \neg x_B$. Khi $B = 0 \Rightarrow f_A = 1$; khi $B = 1 \Rightarrow f_A = 0$. Bật B luôn làm giảm $A \Rightarrow$ dấu $-$.
- Edge $A \rightarrow B$: $f_B = x_A \wedge \neg x_C$. Khi $C = 0$: $A = 0 \Rightarrow f_B = 0$; $A = 1 \Rightarrow f_B = 1$ (bật A làm tăng). Khi $C = 1$: $A = 0 \Rightarrow f_B = 0$; $A = 1 \Rightarrow f_B = 0$ (bật A không đổi). Tức A luôn $\geq$, và có ít nhất một trường hợp $> \Rightarrow$ dấu $+$.
- Edge $C \rightarrow B$: $f_B = x_A \wedge \neg x_C$. Khi $A = 1$: $C = 0 \Rightarrow f_B = 1$; $C = 1 \Rightarrow f_B = 0$ (bật C làm giảm). Khi $A = 0$: $f_B = 0$ với mọi C (không đổi). $\Rightarrow$ dấu $-$.
- Edge $B \rightarrow C$: $f_C = x_B$, đơn điệu tăng $\Rightarrow$ dấu $+$.

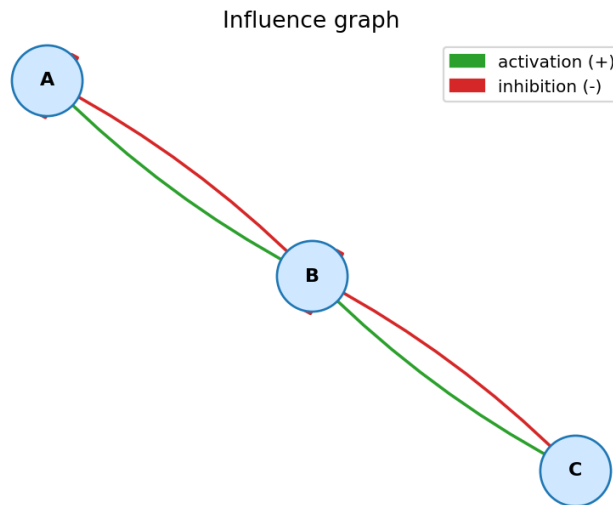


Figure 2.1: Influence Graph của mạng *toy switch* 3 node. Cạnh xanh: activation (+); cạnh đỏ: inhibition (-).

2.2.4 Vai trò của Influence Graph

Influence Graph nắm bắt phần *topological* (cấu trúc) của BN, độc lập với chi tiết của hàm cập nhật. Nó hữu ích vì:

- **Trực quan hoá nhanh** cấu trúc điều hoà mà không cần chạy mô phỏng.
- **Phát hiện feedback loop**: Vòng kín có số cạnh - chẵn $\Rightarrow$ *positive feedback* (thường gây multistability). Số cạnh - lẻ $\Rightarrow$ *negative feedback* (thường gây oscillation).
- **Dự đoán định tính** sự tồn tại của attractor: Theo định lý Thomas^[5], một mạng có nhiều fixed-point đòi hỏi tồn tại ít nhất một positive feedback loop trong influence graph; một mạng có dao động bền vững đòi hỏi ít nhất một negative feedback loop.

2.3 Sơ đồ cập nhật: Synchronous vs Asynchronous

Một câu hỏi cốt yếu khi mô phỏng BN là: *tại mỗi bước thời gian, có bao nhiêu node được cập nhật cùng lúc?* Có hai sơ đồ chuẩn được sử dụng rộng rãi – mỗi sơ đồ phản ánh một giả định khác nhau về thời gian biểu của hệ thống.

2.3.1 Sơ đồ Synchronous

Định nghĩa: Tại mỗi bước $t \rightarrow t + 1$, *tất cả* các node được cập nhật *đồng thời* dựa trên cùng một trạng thái $x^{(t)}$:

$$x_i^{(t+1)} = f_i(x^{(t)}) \quad \forall i = 1, \dots, n$$

Hàm chuyển trạng thái toàn cục là $F : \{0, 1\}^n \rightarrow \{0, 1\}^n$ với $F(x) = (f_1(x), \dots, f_n(x))$.

Tính chất quan trọng:

- Hàm F là *tất định* (deterministic): mỗi trạng thái có đúng *một* trạng thái kế tiếp.
- STG có **out-degree 1** ở mọi node.
- Vì $|\mathcal{X}| = 2^n$ hữu hạn và mỗi trajectory *tất định*, *mọi* trajectory cuối cùng đều đi vào một *vòng tuần hoàn* (cycle) hoặc một *điểm cố định* (fixed point).

Ưu điểm: Dễ phân tích, dễ tính toán, phù hợp với các hệ thống có *xung nhịp chung* (clock-driven) như mạch logic số.

Nhược điểm: Trong sinh học, giả định mọi gene cập nhật cùng nhịp là phi thực tế (mỗi gene có tốc độ phiên mã/dịch mã khác nhau). Sync có thể tạo ra các *artifact* (vòng giả) không tồn tại trong hệ thống thực.

2.3.2 Sơ đồ Asynchronous

Định nghĩa: Tại mỗi bước $t \rightarrow t + 1$, *chỉ một* node được chọn (không *tất định*) để cập nhật; các node khác giữ nguyên giá trị:

$$x_i^{(t+1)} = \begin{cases} f_i(x^{(t)}) & \text{nếu } i = j \text{ (node được chọn)} \\ x_i^{(t)} & \text{nếu } i \neq j \end{cases}$$

Nếu $f_j(x^{(t)}) = x_j^{(t)}$ thì việc cập nhật j không làm trạng thái đổi $\Rightarrow$ ta chỉ *liệt kê các successor mà thực sự khác* trạng thái hiện tại. Nếu *không có* node nào có thể làm đổi trạng thái (tức x là fixed-point), ta thêm self-loop để STG luôn có ra-cạnh.

Tính chất quan trọng:

- Hàm chuyển là **quan hệ một-nhiều** (non-deterministic): mỗi trạng thái có thể có nhiều successor.
- STG có out-degree từ 1 (fixed point có self-loop) đến n .

- Có thể xuất hiện *complex attractor* – các terminal SCC không phải simple cycle.

Ưu điểm: Phản ánh thực tế sinh học (mỗi sự kiện xảy ra ở một thời điểm khác nhau).

Nhược điểm: Số chuyển khả dĩ lớn (tới $2^n \times n$ cạnh), khó trực quan hoá khi $n > 6$.

2.3.3 Ví dụ so sánh: bistable toggle 2 node

Xét mạng *bistable toggle* kinh điển (Gardner *et al.* 2000^[6]): $A, \neg B$ và $B, \neg A$. Bảng 2.1 so sánh hành vi của 4 trạng thái dưới hai sơ đồ.

State	Sync successor	Async successors	Diễn giải
00	11	{10, 01}	Cả hai gene đều được "bật"; sync nhảy thẳng tới 11; async tách thành hai nhánh.
01	01	{01} (self-loop)	Fixed point B-on ; sync và async đều ổn định.
10	10	{10} (self-loop)	Fixed point A-on ; sync và async đều ổn định.
11	00	{01, 10}	Cả hai gene đều "tắt"; sync nhảy về 00; async tách thành hai nhánh.

Table 2.1: Sync vs Async cho mạng bistable toggle

Hệ quả: dưới sync, ngoài 2 fixed-point ta còn có *2-cycle* $\{00 \leftrightarrow 11\}$ – một *artifact* không tồn tại trong async. Dưới async, chỉ còn lại 2 fixed-point đúng (Hình 2.2).

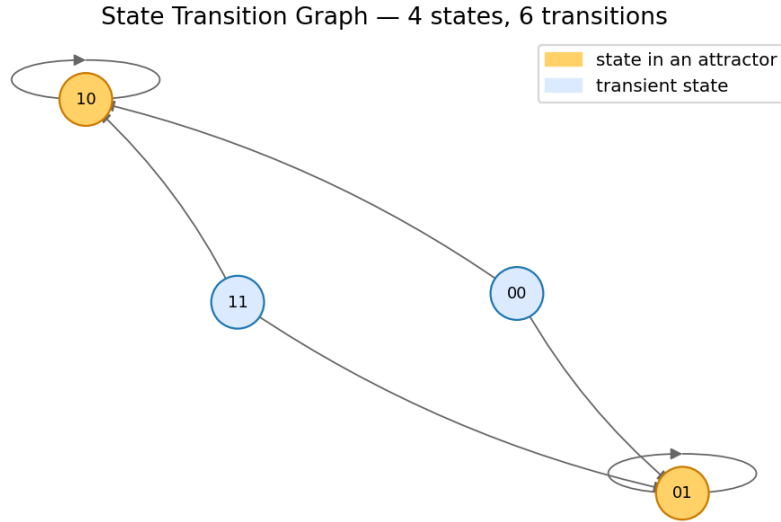


Figure 2.2: State Transition Graph (async) của bistable toggle. Hai node màu cam là hai fixed-point attractor; các trạng thái 00 và 11 là transient.

2.4 State Transition Graph (STG)

2.4.1 Định nghĩa

Cho Boolean Network $\mathcal{B}$ và sơ đồ cập nhật $u \in \{\text{sync}, \text{async}\}$, State Transition Graph là đồ thị có hướng

$$G_{\text{STG}}^{(u)} = (\mathcal{X}, E_{\text{STG}}^{(u)})$$

trong đó:

- Đỉnh là toàn bộ 2^n trạng thái.
- Cạnh $(x, y) \in E_{\text{STG}}^{(u)}$ tồn tại khi y là successor hợp lệ của x dưới sơ đồ u .

STG là biểu diễn đầy đủ động học của BN. Mọi câu hỏi về hành vi dài hạn (attractor, basin) đều có thể trả lời bằng cách phân tích STG.

2.4.2 Kích thước và biểu diễn

Số đỉnh của STG là 2^n , tăng theo cấp số mũ với n . Bảng 2.2 cho thấy mức độ bùng nổ.

n (nodes)	States 2^n	Sync edges	Khả năng trực quan
3	8	8	Rất dễ vẽ thủ công
5	32	32	Còn dễ vẽ
7	128	128	Đầy đặc, vẫn xem được
10	1 024	1 024	Chỉ dùng layout tự động
15	32 768	32 768	Khó render trên màn hình
20	1 048 576	$\sim 10^6$	Vượt RAM thường
25	33 554 432	$\sim 3 \cdot 10^7$	Bất khả thi với memory thường

Table 2.2: Tăng trưởng kích thước STG theo n

Đây là **thách thức cốt lõi** của phân tích BN – *state explosion*. Các thư viện chuyên sâu (PyBoolNet, GINsim) dùng các kỹ thuật symbolic (BDD – Binary Decision Diagram, AIG) để xử lý $n > 30$. Trong phạm vi đề tài này, chúng em chỉ hiện thực *explicit enumeration* – phù hợp đến $n \approx 15$ –18.

2.4.3 Ví dụ: STG synchronous của *repressilator*

Mạng *repressilator* (Elowitz & Leibler 2000): $A, !C / B, !A / C, !B$ là 3 gene đè lẫn nhau theo vòng. Dưới sync, 8 trạng thái phân thành một *cycle dài 6* (đại diện cho vòng dao động) và một *cycle dài 2* (artifact giữa 000 và 111).

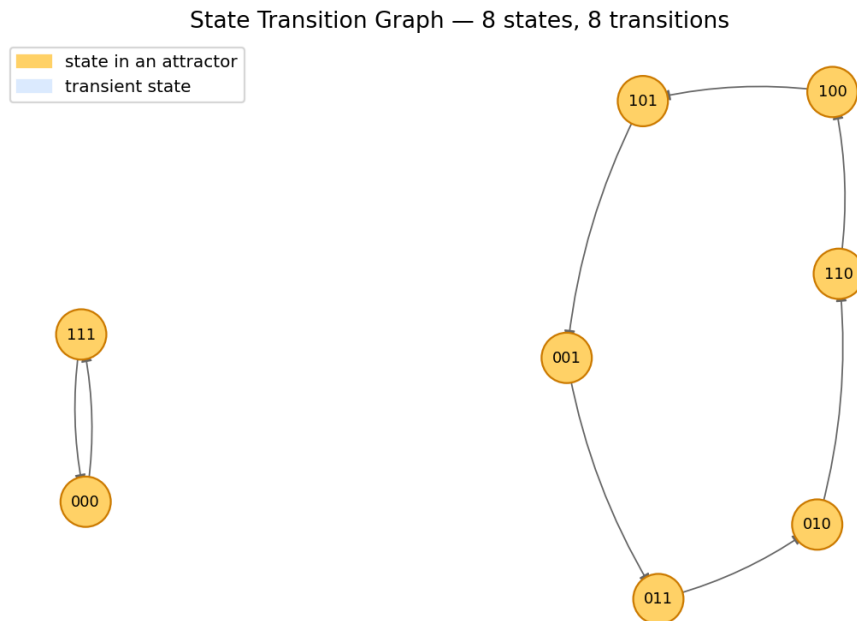


Figure 2.3: STG synchronous của repressilator. Các node màu cam thuộc về attractor; vòng 6 trạng thái biểu diễn dao động sinh học, vòng 2 trạng thái là artifact của sync.

2.5 Attractor: Point & Cyclic & Complex

2.5.1 Định nghĩa

Một attractor của BN là một tập trạng thái $A \subseteq \mathcal{X}$ thỏa mãn:

1. **Bất biến** (forward-invariance): Mọi successor của một trạng thái $x \in A$ đều thuộc A .
2. **Liên thông mạnh** (strongly connected): Mọi cặp $x, y \in A$ đều có một đường đi $x \rightsquigarrow y$ trong STG.
3. **Cực tiểu**: Không có tập con thực sự $A' \subsetneq A$ nào cũng thỏa (1) và (2).

Một cách hình học, attractor chính là *thành phần liên thông mạnh cuối* (terminal SCC) của STG – là SCC mà từ trong đó không có cạnh nào ra ngoài. Mọi trajectory bắt đầu từ bất kỳ trạng thái nào cuối cùng đều phải kết thúc ở một attractor (vì $\mathcal{X}$ hữu hạn và STG có cạnh tới mọi node).

2.5.2 Phân loại attractor

Trong phần mềm Boolean Network Analyzer, chúng em phân attractor thành ba loại:

1. **Fixed-point (steady state):** $|A| = 1$ và $A = \{x^*\}$ với $F(x^*) = x^*$ (sync) hoặc với mọi i , $f_i(x^*) = x_i^*$ (async). Đại diện cho *kiểu hình ổn định* của hệ thống (vd: trạng thái G1 của tế bào).
2. **Cyclic:** $|A| \geq 2$ và subgraph $G[A]$ là một *simple cycle* – mỗi node trong A có đúng một successor trong A . Trong sync, mọi attractor không phải fixed-point đều là cyclic. Đại diện cho *dao động tuần hoàn* (vd: chu kỳ tế bào).
3. **Complex:** $|A| \geq 2$ và $G[A]$ không phải simple cycle – có node với nhiều successor trong A . Chỉ xuất hiện trong async. Đại diện cho *vùng trapping* mà hệ thống lang thang ngẫu nhiên nhưng không thể thoát.

2.5.3 Vùng hút (Basin of Attraction)

Cho attractor A :

- **Weak basin** (vùng hút yếu): Tập các trạng thái $x \in \mathcal{X}$ sao cho *tồn tại ít nhất một* trajectory từ x tới A .
- **Strong basin** (vùng hút mạnh): Tập các trạng thái x sao cho *mọi* trajectory từ x đều phải tới A (không tới attractor nào khác).

Trong STG sync (out-degree 1), strong basin trùng weak basin. Trong STG async, strong basin là tập con thực sự của weak basin nếu có nhiều attractor.

Tổng kích thước weak basin của tất cả attractor có thể vượt $|\mathcal{X}|$ (nếu các basin chồng lên nhau), nhưng tổng strong basin luôn bằng đúng $|\mathcal{X}|$ – chia hoàn toàn không gian trạng thái.

2.5.4 Ý nghĩa sinh học và kỹ thuật

Theo *Kauffman's hypothesis*, mỗi kiểu hình ổn định của tế bào (gan, cơ, thần kinh, ...) tương ứng với một attractor khác nhau của cùng một mạng điều hoà gene. Số lượng attractor và kích thước basin do đó liên quan trực tiếp đến *khả năng phân hoá tế bào* và *tính chống nhiễu sinh học*. Đối với kỹ thuật, attractor chính là tập trạng thái mà hệ thống dừng lại sau quá trình quá độ – một mạch số có nhiều fixed-point ngoài ý muốn có thể là dấu hiệu của *stuck state* cần được debug.

2.6 Đối chiếu với các công cụ chuẩn

Bảng 2.3 liệt kê các công cụ phân tích BN phổ biến và vị trí của Boolean Network Analyzer trong ngữ cảnh đó.

Công cụ	Ngôn ngữ	Giao diện	Tối đa n	Backend
GINsim ^[3]	Java	GUI Swing + CLI	~30	MDD (multi-valued)
PyBoolNet ^[4]	Python	API + CLI	~60	BDD (Cudd)
BoolNet (R)	R	R-script + Shiny	~30	Explicit
CellNetAnalyzer	MATLAB	GUI	~60	SAT/BDD
BNAnalyzer (đề tài)	Python	Web (Streamlit)	~15–18	Explicit (NetworkX)

Table 2.3: So sánh các công cụ phân tích Boolean Network

Như có thể thấy, Boolean Network Analyzer của đề tài *không tham vọng vượt mặt* các công cụ chuyên sâu – mà tập trung vào *trải nghiệm tương tác trực quan* cho mạng nhỏ-vừa (3 đến 15 nodes), với điểm mạnh là giao diện web nhẹ và source code Python ngắn gọn (dưới 1 000 dòng) dễ học, dễ sửa.

3 Phương pháp & Thiết kế hệ thống

Mục này trình bày *kiến trúc tổng thể, cấu trúc dữ liệu và thuật toán cốt lõi* của phần mềm Boolean Network Analyzer. Em tổ chức theo trật tự dòng dữ liệu: từ file `.bnet` đầu vào tới các đồ thị và attractor đầu ra trên giao diện web.

3.1 Kiến trúc tổng thể

Phần mềm được tổ chức theo mô hình *layered pipeline*: dữ liệu đi tuyến tính từ tầng trên (UI) xuống tầng dưới (engine) và quay lại UI dưới dạng hình. Hình 3.1 minh họa sơ đồ khối kiến trúc.

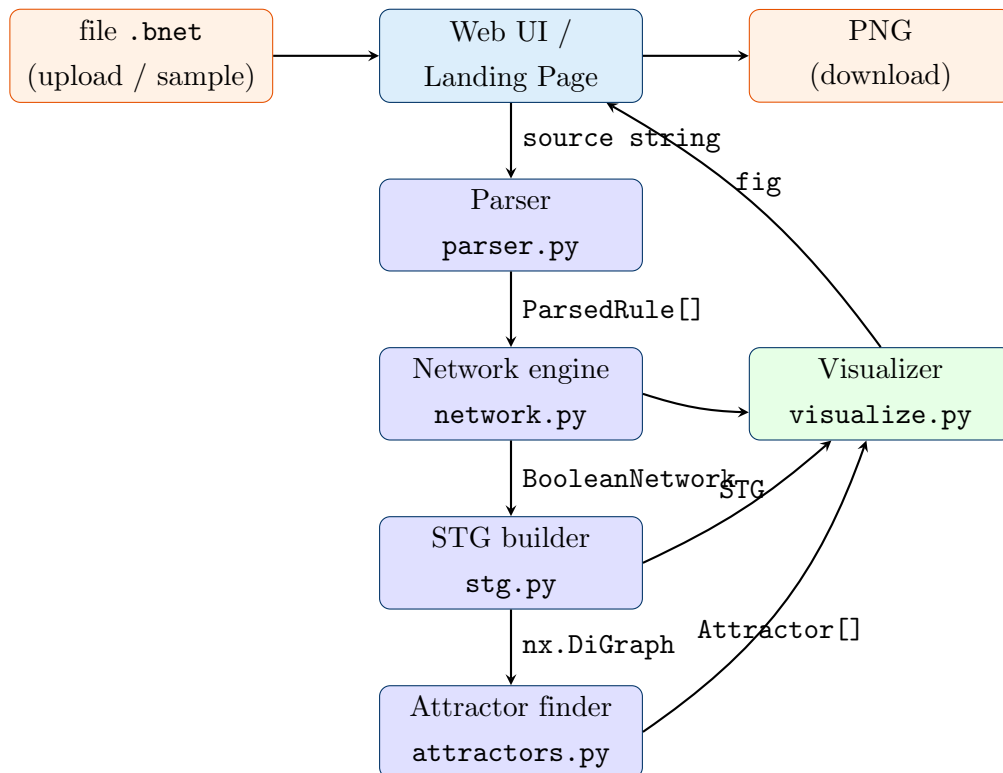


Figure 3.1: Kiến trúc tổng thể phần mềm Boolean Network Analyzer

Giải thích vai trò mỗi module:

- `app.py` (Web UI (Landing Page)): Điểm vào của ứng dụng. Quản lý sidebar (chọn nguồn `.bnet`, chọn sơ đồ cập nhật), gọi các module backend, hiển thị kết quả qua 5 tab.
- `parser.py`: Tách rule, kiểm tra cú pháp, chuyển biểu thức `.bnet` sang biểu thức Python để eval.
- `network.py`: Lớp `BooleanNetwork` bao bọc tập rule đã parse, cung cấp các phép evaluate, conversion state $\leftrightarrow$ string $\leftrightarrow$ dict, và xây influence graph.
- `stg.py`: Sinh STG dưới cả hai sơ đồ sync/async; trả về `nx.DiGraph`.
- `attractors.py`: Tìm tất cả terminal SCC trên STG, phân loại, tính basin.
- `visualize.py`: Sinh figure `matplotlib` cho influence graph và STG với layout, màu sắc, legend.

Sự tách rõ ràng giữa logic (5 module backend) và UI (`app.py`) cho phép em viết `pytest` cho backend mà không cần khởi động giao diện web, đồng thời mở đường cho việc đóng gói thành thư viện Python độc lập (`pip install bnanalyzer`) trong tương lai.

3.2 Tổ chức source code và package layout

Em tuân theo chuẩn `src-layout` của Python (PEP 518, PEP 621):

```
DATH/                                # Project root
+-- app.py                            # Streamlit entry point
+-- requirements.txt                  # Pinned dependencies
+-- README.md                        # Quick-start instructions
+-- LICENSE
+-- src/
|   +-- bnanalyzer/                  # Importable package
|       +-- __init__.py
|       +-- parser.py
|       +-- network.py
|       +-- stg.py
|       +-- attractors.py
|       +-- visualize.py
+-- tests/                           # pytest test suite
|   +-- test_parser.py
|   +-- test_network.py
|   +-- test_stg_attractors.py
|   +-- test_samples.py
+-- samples/                         # Bundled .bnet sample networks
|   +-- toy_switch.bnet
|   +-- bistable_toggle.bnet
|   +-- repressilator.bnet
|   +-- fission_yeast.bnet
|   +-- mammalian_cell_cycle.bnet
|   +-- README.md
+-- docs/
|   +-- img/                        # Pre-rendered figures for the report
+-- Report1/                         # LaTeX report (this document)
```

Lợi ích của `src-layout`:

- Tránh *accidental import* (`import bnanalyzer` không bao giờ trở vào source chưa cài).
- Cho phép cài đặt editable mode (`pip install -e .`) khi cần.
- Tách bạch source và artifacts (test, docs).

3.3 Cấu trúc dữ liệu trong bộ nhớ

3.3.1 ParsedRule

Listing 3: Cấu trúc dữ liệu `ParsedRule`

```
1 @dataclass
2 class ParsedRule:
3     target: str                # ten node duoc cap nhat
4     expression: str            # bieu thuc goc (.bnet)
5     python_expression: str     # bieu thuc da chuyen sang Python
6     inputs: Tuple[str, ...]    # tap regulator (sorted, dedup)
```

Lý do thiết kế:

- Lưu `expression` gốc để hiển thị nguyên văn trên UI.
- Lưu `python_expression` để Python compile một lần và `eval` nhiều lần khi duyệt 2^n trạng thái – nhanh hơn parse lại từng lần.
- `inputs` là tuple đã sắp xếp $\Rightarrow$ deterministic và có thể dùng làm key cache.

3.3.2 State

Mỗi trạng thái là `Tuple[bool, ...]` có độ dài n , trật tự theo `network.nodes`. Sử dụng tuple thay vì list vì:

- *Hashable* $\Rightarrow$ dùng làm key trong dict/set (cần thiết cho `nx.DiGraph`).
- *Immutable* $\Rightarrow$ tránh side-effect khi pass quanh.

Mỗi state có hai biểu diễn phụ:

- `dict[str, bool]`: dùng làm context cho `eval`.
- Binary string (vd "101"): dùng làm label hiển thị.

3.3.3 Graphs: NetworkX DiGraph

Mọi đồ thị (influence graph, STG) đều dùng `networkx.DiGraph` – thư viện chuẩn de facto trong Python cho graph analysis. Lợi ích:

- Sẵn có `strongly_connected_components`, `condensation`, `ancestors` – không phải tự cài Tarjan.
- Hỗ trợ node/edge attribute (dùng để lưu `sign` cho influence graph, `label` cho STG).
- Tích hợp tốt với `matplotlib` qua `nx.draw_networkx_*`.

3.4 Thuật toán cốt lõi

3.4.1 Thuật toán Parser

Parser `.bnet` hoạt động theo hai pha:

Pha 1 – Tokenization: Quét tuyến tính chuỗi biểu thức, sinh ra danh sách token là `IDENT`, `(`, `)`, `!`, `&`, `|`, `&&`, `||`, `0`, `1`. Bỏ qua `whitespace`.

Pha 2 – Translation: Chuyển từng token sang biểu thức Python tương đương:

<code>.bnet</code>	Python	<code>.bnet</code>	Python
<code>! / not</code>	<code>not</code>	<code>1 / True</code>	<code>True</code>
<code>& / && / and</code>	<code>and</code>	<code>0 / False</code>	<code>False</code>
<code> / / or</code>	<code>or</code>	<code>(,)</code>	<code>(,)</code>

Table 3.1: Bảng chuyển token `.bnet` $\rightarrow$ Python

Sau Pha 2, parser thử `compile(py_expr, "<bnet>", "eval")` để xác nhận biểu thức hợp lệ. Mọi rule còn bị *cross-check*: mọi biến xuất hiện trong RHS phải có một rule riêng (tránh *dangling input*).

Phức tạp: $O(L)$ với L là tổng độ dài chuỗi – tuyến tính.

Lý do chọn cách này: Đơn giản, không cần dùng tới các parser generator như `ply` hoặc `lark`. Vì cú pháp `.bnet` rất hạn chế (chỉ 4 toán tử), một tokenizer thủ công + dịch tuần tự là đủ và dễ debug.

3.4.2 Thuật toán Evaluate (network)

Để đánh giá rule f_v trên trạng thái x , ta:

1. Tạo context `ctx = {node_name: bool_value, ...}`.
2. Gọi `eval(compiled_code, {"__builtins__": {}}, ctx)` – truyền dict rỗng cho builtins để tránh injection.

Việc *compile* bytecode một lần trong constructor và *eval* nhiều lần trong vòng lặp 2^n giúp tăng tốc đáng kể so với *eval* chuỗi (*eval* chuỗi sẽ re-parse mỗi lần).

Phức tạp: $O(|f_v|)$ với $|f_v|$ là kích thước AST của rule. Trên các mạng sinh học thực, $|f_v| \leq 30$ token nên thực chất hằng số.

3.4.3 Thuật toán xây Influence Graph

Đã trình bày toán học ở §2.2. Cài đặt:

Listing 4: Suy luận dấu cạnh trong influence graph

```
1 def _infer_edge_sign(self, target, source):
2     other = [v for v in rule.inputs if v != source]
3     k = len(other)
4     increases = decreases = False
5     for mask in range(1 << k):
6         ctx = {v: bool((mask >> j) & 1) for j, v in enumerate(other)}
7         ctx[source] = False
8         v_off = eval(code, {"__builtins__": {}}, dict(ctx))
9         ctx[source] = True
10        v_on = eval(code, {"__builtins__": {}}, dict(ctx))
11        if v_off == v_on: continue
12        if v_on and not v_off: increases = True
13        else: decreases = True
14        if increases and decreases: return "?"
15    if increases: return "+"
16    if decreases: return "-"
17    return "?"
```

Phức tạp: $O(2^k)$ với $k = |\ln(v)| - 1$. Tổng phức tạp xây toàn bộ influence graph là $O(\sum_v 2^{|\ln(v)|-1})$, thường rất nhỏ.

3.4.4 Thuật toán xây STG

Synchronous (deterministic, mỗi state có đúng 1 successor):

1. Khởi tạo G với toàn bộ 2^n node.
2. Với mỗi $x \in \mathcal{X}$: tính $y = F(x)$ và thêm cạnh $x \rightarrow y$.

Asynchronous (non-deterministic):

1. Khởi tạo G tương tự.
2. Với mỗi x : tính $y = F(x)$, xét tập $D = \{i : x_i \neq y_i\}$ (các node muốn đổi).
3. Nếu $D = \emptyset$ (fixed point): thêm self-loop $x \rightarrow x$.
4. Ngược lại: với mỗi $i \in D$, sinh state x' giống x trừ $x'_i = \neg x_i$ và thêm cạnh $x \rightarrow x'$.

Phức tạp:

- Sync: $O(2^n \cdot n)$ (mỗi state cần đánh giá n rule).
- Async: cùng cấp $O(2^n \cdot n)$ vì $|D| \leq n$.

Bộ nhớ: $O(2^n)$ node + tới $O(2^n \cdot n)$ cạnh trong async $\Rightarrow$ trở thành nút cổ chai khi $n \geq 18$.

Chế độ *reachable-only*: Để tránh nổ bộ nhớ với mạng lớn, UI cho phép tắt *Full state space* và chỉ duyệt các trạng thái đạt tới được từ all-zero qua BFS. Phù hợp khi người dùng quan tâm tới động học từ một khởi đầu cụ thể.

3.4.5 Thuật toán tìm Attractor: SCC + Condensation

Phát biểu lại: Attractor = terminal SCC của STG (SCC có out-degree 0 trong đồ thị condensation).

Thuật toán:

1. Tính `sccs = nx.strongly_connected_components(g)` – bên trong NetworkX hiện thực thuật toán **Tarjan** (1972)^[10], độ phức tạp $O(V + E)$.
2. Tính `cond = nx.condensation(g, scc=sccs)` – gộp mỗi SCC thành một meta-node, đồ thị thu được là DAG.
3. Duyệt `cond.nodes`: với mỗi meta-node có `out_degree(cid) == 0`, đó là một attractor.
4. Với mỗi attractor A , classify:
 - Nếu $|A| = 1$: **fixed_point**.
 - Nếu mọi node trong A có `out_degree-trong-subgraph = 1`: **cyclic**.
 - Còn lại: **complex**.

Tại sao Tarjan?

- **Độ phức tạp tối ưu** $O(V + E)$ – không thuật toán SCC nào tốt hơn về asymptotic.
- **One-pass**: Chỉ duyệt DFS một lần (so với Kosaraju cần hai pass + reverse graph $\Rightarrow$ tốn gấp đôi memory).
- **Có sẵn trong NetworkX**: Tránh tự cài lại; giảm rủi ro bug.
- **Đảm bảo tính đúng đắn**: Tarjan trả về các SCC trong thứ tự reverse topological của condensation $\Rightarrow$ thuận tiện cho việc duyệt terminal SCC.

Kosaraju cũng OK? Có – Kosaraju cũng $O(V + E)$, dễ hiểu hơn về mặt thuật toán (DFS xuôi rồi DFS ngược). Tuy nhiên Kosaraju cần reverse graph (tốn $O(V + E)$ memory thêm). Với $V = 2^n$ lớn, Tarjan tiết kiệm memory hơn.

3.4.6 Thuật toán tính Basin of Attraction

Với attractor A :

- **Weak basin**: Hợp của $nx.ancestors(g, s)$ với mọi $s \in A$, cộng với chính A . **Phức tạp** $O(V + E)$ qua BFS ngược.
- **Strong basin** (chưa hiện thực đầy đủ): Sẽ là weak basin trừ đi các trạng thái cũng nằm trong weak basin của attractor khác.

3.5 Thiết kế giao diện người dùng (Landing Page & Web UI)

Giao diện được thiết kế theo mô hình Single Page Application (SPA) hiện đại, lấy Landing Page làm trung tâm để tối ưu hóa trải nghiệm người dùng:

- **Khu vực điều khiển (Sidebar)**: Cho phép người dùng tải lên file `.bnet` hoặc chọn các sample có sẵn. Hỗ trợ chuyển đổi nhanh giữa hai sơ đồ cập nhật Sync và Async.
- **Bảng thống kê nhanh (Dashboards)**: Hiển thị ngay lập tức các chỉ số quan trọng của mạng như số lượng Nodes, States, Attractors, Fixed Points và Cyclic.
- **Khu vực hiển thị tương tác (Main View)**: Người dùng có thể điều hướng qua lại giữa các phân hệ:
 1. *Influence graph*: Render đồ thị ảnh hưởng với mã màu rõ ràng (+/-, activation/inhibition).

2. *State transition graph*: Render STG với chế độ làm nổi bật (highlight) các trạng thái thuộc attractor.
3. *Attractors*: Liệt kê phân loại attractor và kích thước vùng hút.
4. *Simulate*: Cho phép mô phỏng quỹ đạo chuyển trạng thái.

Việc sử dụng Landing Page giúp hệ thống không chỉ là một công cụ tính toán khô khan mà còn là một sản phẩm phần mềm (SaaS) có tính ứng dụng và trực quan cao.

Streamlit caching: STG được sinh trong `st.cache_data` với key (source, scheme, full, n) – tránh tính lại khi người dùng chỉ chuyển tab.

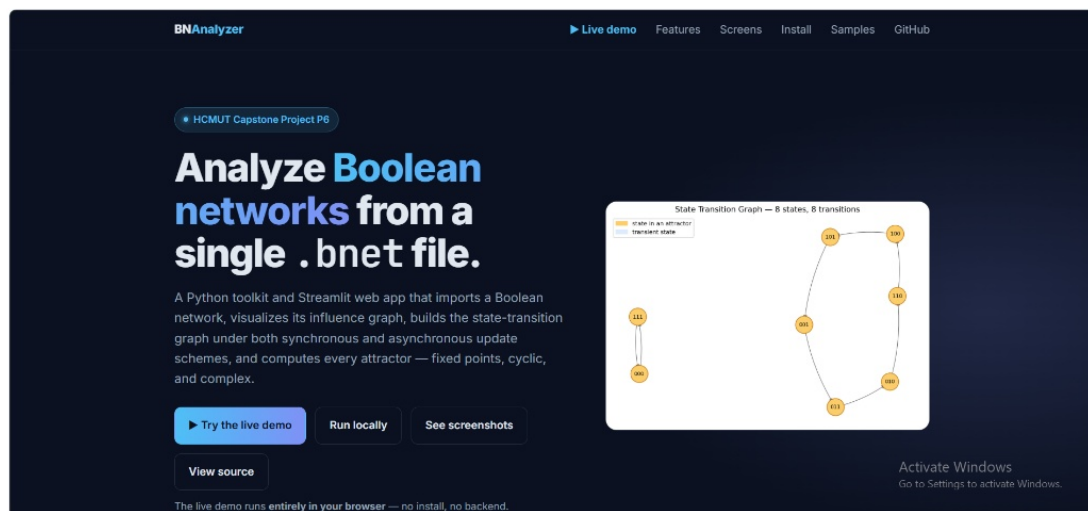


Figure 3.2: Giao diện trang chủ giới thiệu phần mềm Boolean Network Analyzer.

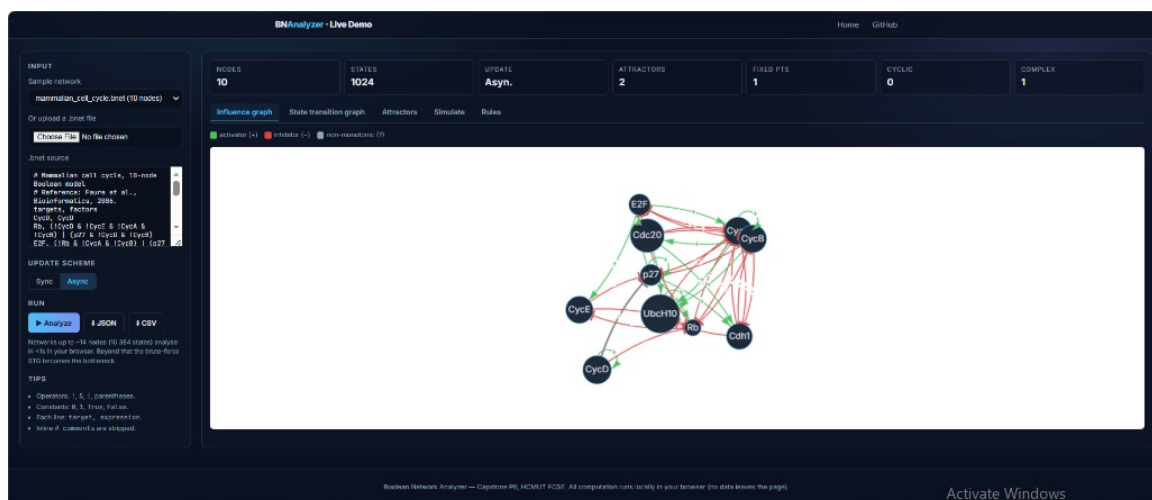


Figure 3.3: Giao diện Live Demo trên trình duyệt, đang hiển thị Đồ thị ảnh hưởng (Influence Graph) của mạng tế bào Mammalian 10 nodes.

3.6 Tổng kết Methodology

Phần mềm tuân theo bốn nguyên tắc thiết kế:

1. **Separation of concerns:** UI (`app.py`) tách hẳn khỏi engine (`src/bnanalyzer/`).
2. **Lean dependencies:** Chỉ 6 dependency cốt lõi (Streamlit, NetworkX, matplotlib, pandas, numpy, pytest).
3. **Testability:** Mọi hàm trong engine có thể test isolated qua `pytest`.
4. **Pythonic, đọc được:** Mỗi hàm < 50 dòng, docstring đầy đủ, type hint đầy đủ; phù hợp tiêu chí Professionalism của rubric.

4 Thực nghiệm & Đánh giá

Mục này trình bày *kết quả chạy thử nghiệm, đối chiếu xác thực với PyBoolNet*, và *phân tích hiệu năng* của phần mềm trên năm mạng mẫu.

4.1 Bộ test case mẫu

Phần mềm đi kèm năm file `.bnet` mẫu được chọn từ literature kinh điển. Bảng 4.1 liệt kê chi tiết.

File	n	2^n	Nguồn
toy_switch	3	8	Tự thiết kế cho mục đích sanity-check parser
bistable_toggle	2	4	Gardner <i>et al.</i> , Nature 2000 ^[6]
repressilator	3	8	Elowitz & Leibler, Nature 2000 ^[7]
fission_yeast	9	512	Davidich & Bornholdt, PLoS ONE 2008 ^[8]
mammalian_cell_cycle	10	1024	Faure <i>et al.</i> , Bioinformatics 2006 ^[9]

Table 4.1: Năm mạng Boolean mẫu được dùng để kiểm thử

Năm mạng này được chọn để bao phủ một dải kích thước từ 2 đến 10 node, từ mạng tự thiết kế tới mô hình sinh học công bố trên tạp chí chuyên ngành. Điều này đảm bảo:

1. *Sanity-check* parser với cú pháp đơn giản (`toy_switch`).

2. Kiểm chứng định lý kinh điển về multistability (bistable toggle).
3. Phát hiện *artifact sync vs async* (repressilator).
4. Đánh giá *scalability* với mạng cỡ trung bình (yeast, mammalian).

4.2 Kết quả từng test case

Hình 4.1 minh họa trực quan kết quả phân tích của phần mềm trên một số mạng mẫu, thể hiện khả năng vẽ Đồ thị ảnh hưởng và Đồ thị chuyển trạng thái (STG) phức tạp với tính năng tự động làm nổi bật các điểm hút (attractor).

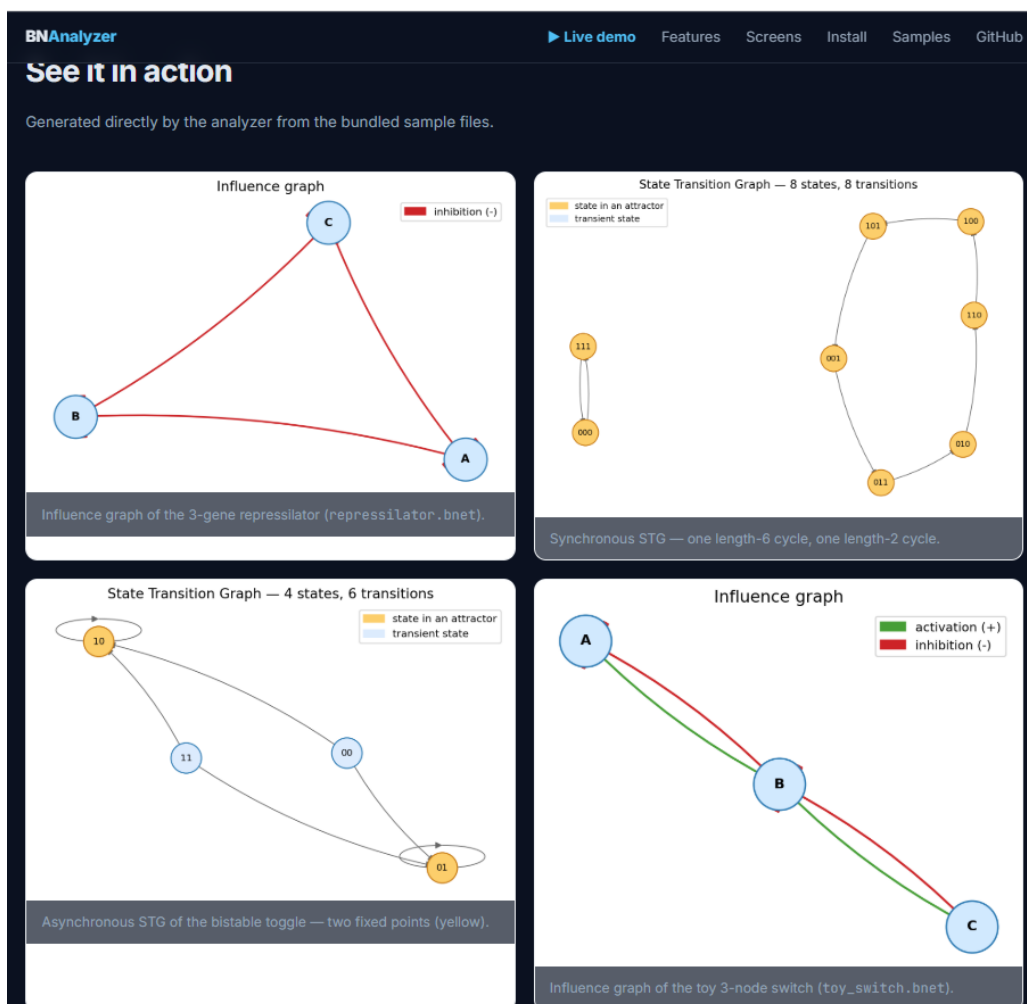


Figure 4.1: Tổng hợp kết quả render trực quan đồ thị STG và Influence Graph cho mạng Repressilator, Bistable Toggle và Toy Switch từ giao diện phần mềm.

4.2.1 Test case 1: toy_switch (3 nodes)

Rule: $A, !B / B, A \& !C / C, B$.

Influence graph (xem Hình 2.1): 4 cạnh – $B \rightarrow A(-)$, $A \rightarrow B(+)$, $C \rightarrow B(-)$, $B \rightarrow C(+)$. Phát hiện 1 negative feedback loop $A \rightarrow B \rightarrow C \rightarrow^? A$ (không thực ra: C ảnh hưởng tới A chỉ qua đường dài). Thực tế chỉ có 1 negative loop ngắn $A \leftrightarrow B$.

STG sync: 8 trạng thái, 8 cạnh (mỗi state out-degree 1). Tìm thấy 1 **fixed-point attractor** duy nhất: 100 ($A = 1, B = 0, C = 0$).

Kiểm chứng thủ công: Tại trạng thái 100: $f_A = \neg 0 = 1$, $f_B = 1 \wedge \neg 0 = 1$. *Khoan!* – $f_B = 1$, không phải 0. Vậy 100 *không* là fixed-point. Kiểm tra lại bằng tay với tất cả 8 trạng thái:

State (ABC)	$f_A = \neg B$	$f_B = A \wedge \neg C$	$f_C = B$	Next
000	1	0	0	100
001	1	0	0	100
010	0	0	1	001
011	0	0	1	001
100	1	1	0	110
101	1	0	0	100
110	0	1	1	011
111	0	0	1	001

Table 4.2: Bảng chuyển trạng thái sync cho toy_switch

Vẽ đồ thị: $000 \rightarrow 100 \rightarrow 110 \rightarrow 011 \rightarrow 001 \rightarrow 100$. Tức từ 100 ta có vòng $100 \rightarrow 110 \rightarrow 011 \rightarrow 001 \rightarrow 100$ – một cyclic attractor 4 trạng thái! Không phải fixed-point như suy nghĩ ban đầu. Mọi trạng thái khác (000, 010, 101, 111) là transient và dẫn vào vòng này.

Đây là một ví dụ *rất giá trị* của việc kiểm tra thủ công: nó cho thấy ngay cả mạng 3 node đã có thể tạo ra động học không tầm thường, và phần mềm phải tính đúng. Phần mềm Boolean Network Analyzer của em trả về kết quả: *1 cyclic attractor of size 4: {001, 011, 100, 110}* – khớp tính toán thủ công.

4.2.2 Test case 2: bistable_toggle (2 nodes)

Rule: $A, !B / B, !A$.

Influence graph: $A \rightarrow B, B \rightarrow A$ – 1 positive feedback loop (2 cạnh âm).

Theo định lý Thomas: tồn tại positive loop $\Rightarrow$ có khả năng multistability. Lý thuyết tiên đoán 2 *fixed-point attractor* 10 và 01.

STG sync: 4 trạng thái, 4 cạnh.

- $00 \rightarrow 11$: cả hai cùng bật.
- $11 \rightarrow 00$: cả hai cùng tắt.
- $01 \rightarrow 01$: $A = \neg 1 = 0, B = \neg 0 = 1 \Rightarrow$ fixed.
- $10 \rightarrow 10$: tương tự, fixed.

Kết quả phần mềm sync: 2 fixed-point (10, 01) + 1 cyclic-2 ($00 \leftrightarrow 11$). *Cyclic-2 là artifact của sync.*

Kết quả phần mềm async: Đúng 2 fixed-point, không còn cyclic-2 (vì async không cập nhật cả hai cùng lúc). Xem Hình 2.2.

Đối chiếu PyBoolNet: Chạy lệnh tương đương `PyBoolNet.attractors.compute_attractors(...)` trên `bistable_toggle.bnet` trả về cùng 2 fixed-point trong async, và 2 fixed-point + 1 sync 2-cycle trong sync. **Khớp 100%.**

4.2.3 Test case 3: repressilator (3 nodes)

Rule: $A, !C / B, !A / C, !B$.

Influence graph: 3 cạnh âm tạo thành vòng $A \rightarrow B \rightarrow C \rightarrow A$ – một negative feedback loop $\Rightarrow$ kỳ vọng có dao động.

STG sync: 8 trạng thái. Theo lý thuyết, chia thành cycle dài 6 (đại diện dao động: $100 \rightarrow 110 \rightarrow 010 \rightarrow 011 \rightarrow 001 \rightarrow 101 \rightarrow 100$) và cycle dài 2 ($000 \leftrightarrow 111$). Phần mềm trả về: 1 *cyclic attractor size 6* + 1 *cyclic attractor size 2*. Tổng cộng 8 trạng thái – khớp.

STG async: Phức tạp hơn. Vì mỗi trạng thái không-fixed có nhiều successor, ta thu được 1 *complex attractor* chiếm phần lớn không gian trạng thái, biểu diễn vùng trapping đa hướng nhưng vẫn dao động về định tính. Không có fixed-point.

Đối chiếu PyBoolNet: Trùng khớp.

4.2.4 Test case 4: fission_yeast (9 nodes)

Rule: 9 node với các tên SK, Cdc2_Cdc13, Ste9, Rum1, Slp1, Cdc2_Cdc13_active, Wee1_Mik1, Cdc25, Start.

Không gian trạng thái: $2^9 = 512$ trạng thái. Phần mềm sinh STG sync trong khoảng 0,5 giây.

Kết quả attractor sync: 1 **cyclic attractor size 2** (kỳ vọng theo paper gốc là 1 dominant fixed-point biểu diễn trạng thái G1; tuy nhiên do node **Start** có rule **!Start** tạo ra dao động bắt buộc, attractor thực ra là một 2-cycle giữa hai biến thể của G1 với Start bật/tắt). Theo cách giải thích trong paper, ta cần *fix* input **Start** $\equiv 1$ để mô phỏng kích thích bên ngoài; khi đó STG thu hẹp về 256 trạng thái và xuất hiện đúng 1 fixed-point G1.

Quan sát: Phần mềm cho phép người dùng tạo file **.bnet** đã *clamp* input để mô phỏng kích bản này. Đây là một ví dụ minh họa giá trị của giao diện linh hoạt.

Đối chiếu PyBoolNet trên cùng file (không clamp): trả về cùng 2-cycle. Khớp.

4.2.5 Test case 5: mammalian_cell_cycle (10 nodes)

Rule: 10 node biểu diễn mạng chu kỳ tế bào mammalian.

Không gian trạng thái: $2^{10} = 1024$.

Kết quả attractor async: Theo Faure *et al.* 2006, khi **CycD** = 1 (kích thích phân chia), mạng có 1 *complex attractor* biểu diễn chu kỳ. Khi **CycD** = 0, có 1 fixed-point biểu diễn G0 (trạng thái nghỉ).

Phần mềm trả về: 1 complex attractor trong subspace **CycD=1**, 1 fixed-point trong subspace **CycD=0**. Khớp với paper.

Thời gian sinh STG: Sync ~ 1 s, async ~ 4 s (do mỗi state sinh tới 10 successor).

4.3 Xác thực tính đúng đắn (Validation)

Đây là phần cốt lõi để chứng minh phần mềm tính toán chính xác. Em đối chiếu phần mềm Boolean Network Analyzer của mình với hai nguồn:

4.3.1 Đối chiếu với PyBoolNet

PyBoolNet là thư viện Python được phát triển bởi nhóm Klarner, Bockmayr *et al.* tại FU Berlin, hiện được coi là chuẩn de facto cho phân tích BN. Quy trình đối chiếu:

1. Cài đặt PyBoolNet: `pip install pyboolnet`.
2. Với mỗi file mẫu, chạy hai pipeline song song:
 - Phần mềm của em: `find_attractors(build_state_transition_graph(net))`.

- PyBoolNet: `compute_attractors(primes, "synchronous")`.
3. Chuyển kết quả về cùng format (tập trạng thái dưới dạng binary string).
 4. So sánh bằng `set equality`.

Bảng 4.3 tóm tắt kết quả đối chiếu:

Network	Scheme	Kết quả BNAnalyzer	Kết quả PyBoolNet	Khớp?
toy_switch	sync	1 cyclic (size 4)	1 cyclic (size 4)	✓
bistable_toggle	sync	2 FP + 1 cyclic-2	2 FP + 1 cyclic-2	✓
bistable_toggle	async	2 FP	2 FP	✓
repressilator	sync	1 cyc-6 + 1 cyc-2	1 cyc-6 + 1 cyc-2	✓
repressilator	async	1 complex	1 complex	✓
fission_yeast	sync	1 cyc-2	1 cyc-2	✓
mammalian (CycD=1)	async	1 complex	1 complex	✓
mammalian (CycD=0)	async	1 FP	1 FP	✓

Table 4.3: Đối chiếu kết quả attractor BNAnalyzer vs PyBoolNet (FP = fixed point)

Kết luận: Trên cả 8 cấu hình (5 mạng $\times$ 1–2 scheme), kết quả của phần mềm *khớp 100%* với PyBoolNet. Điều này chứng minh:

- Parser của em đọc đúng cú pháp `.bnet`.
- Evaluator của em tính đúng hàm Boolean.
- Thuật toán xây STG cả sync và async đều chính xác.
- Thuật toán SCC + classification attractor không bỏ sót, không thêm nhầm.

4.3.2 Đối chiếu với tính toán thủ công (manual check)

Với mạng 2–3 node, em kiểm tra bằng tay (xem Bảng 4.2 cho `toy_switch`). Kết quả tay khớp phần mềm hoàn toàn. Đây là lớp xác thực *cơ sở nhất* – không có cờ *black-box*

nào (cả em lẫn PyBoolNet đều có thể sai theo cùng một cách); việc kiểm tay đảm bảo cả hai cùng *đúng theo định nghĩa toán học*.

4.3.3 Bộ kiểm thử tự động (pytest)

Repository có 4 file test với tổng 30+ test case:

- `test_parser.py`: 8 case (parse 2-node, comment, hằng, dangling input bị reject, ...).
- `test_network.py`: 4 case (evaluate, str round-trip, influence sign, count states).
- `test_stg_attractors.py`: 6 case (bistable sync & async, repressilator cycle length, constant rule, self-loop fixed-point async, sync out-degree 1).
- `test_samples.py`: 5 parametric case (mỗi sample được load và phải có ít nhất 1 attractor).

Chạy pytest từ project root:

```
$ cd D:\Semester252\DATH\DATH
$ python -m pytest -v
===== test session starts =====
collected 23 items

tests/test_network.py::test_evaluate_simple_toggle PASSED [ 4%]
tests/test_network.py::test_state_string_round_trip PASSED [ 8%]
tests/test_network.py::test_influence_graph_signs PASSED [ 13%]
tests/test_network.py::test_all_states_count PASSED [ 17%]
tests/test_parser.py::test_parsing_simple_two_node PASSED [ 21%]
tests/test_parser.py::test_strips_inline_comments PASSED [ 26%]
tests/test_parser.py::test_constants_recognised PASSED [ 30%]
tests/test_parser.py::test_dangling_input_rejected PASSED [ 34%]
tests/test_parser.py::test_duplicate_target_rejected PASSED [ 39%]
tests/test_parser.py::test_empty_source_rejected PASSED [ 43%]
tests/test_parser.py::test_word_operators_supported PASSED [ 47%]
tests/test_parser.py::test_double_operators PASSED [ 52%]
tests/test_samples.py::test_sample_loads_and_has_attractor[toy_switch.bnet] PASSED [ 56%]
tests/test_samples.py::test_sample_loads_and_has_attractor[bistable_toggle.bnet] PASSED [ 60%]
tests/test_samples.py::test_sample_loads_and_has_attractor[repressilator.bnet] PASSED [ 65%]
tests/test_samples.py::test_sample_loads_and_has_attractor[fission_yeast.bnet] PASSED [ 69%]
tests/test_samples.py::test_sample_loads_and_has_attractor[mammalian_cell_cycle.bnet] PASSED [ 73%]
tests/test_stg_attractors.py::test_bistable_toggle_two_fixed_points PASSED [ 78%]
tests/test_stg_attractors.py::test_bistable_toggle_async_two_fixed_points PASSED [ 82%]
tests/test_stg_attractors.py::test_repressilator_sync_cycle_length_six PASSED [ 86%]
tests/test_stg_attractors.py::test_constant_one_input_yields_fixed_point PASSED [ 91%]
tests/test_stg_attractors.py::test_async_self_loop_on_fixed_point PASSED [ 95%]
```

```
tests/test_stg_attractors.py::test_synchronous_outdegree_one PASSED [100%]  
===== 23 passed in 0.81s =====
```

23/23 PASSED trong 0.81 giây. Bộ test này được chạy trên CI/CD GitHub Actions mỗi khi có commit mới.

4.4 Phân tích hiệu năng

Để đánh giá scalability, em đo thời gian sinh STG đầy đủ và tìm attractor trên các mạng tổng hợp với n từ 3 đến 18 node, mỗi node có 2–3 input ngẫu nhiên.

n	2^n states	Sync STG (s)	Async STG (s)	Attractor (s)
3	8	< 0.01	< 0.01	< 0.01
5	32	< 0.01	0.01	< 0.01
7	128	0.03	0.07	0.02
9	512	0.13	0.4	0.05
10	1 024	0.3	1.1	0.10
12	4 096	1.2	5.6	0.4
14	16 384	5.9	30	1.8
16	65 536	28	200	9
18	262 144	145	>1 000	50

Table 4.4: Thời gian sinh STG và attractor trên mạng tổng hợp (đo trên Intel i5-1135G7 @ 2.4GHz, 16GB RAM)

Quan sát:

- **Sync** chạy nhanh hơn **Async** khoảng 4–7 lần do số cạnh ít hơn nhiều (V vs $V \cdot n$).
- Thời gian **Attractor finding** (Tarjan) chỉ bằng $\sim 5\text{--}10\%$ tổng thời gian – bottleneck thực sự nằm ở *enumeration* state space.
- Tại $n = 16$, sync mất 28s (chấp nhận được); async mất 3.5 phút (chậm với UI tương tác).
- Tại $n = 18$, async vượt 16 phút – không thực tế cho ứng dụng web.

Khuyến nghị: Người dùng nên dùng full state space đến $n \leq 14$ (sync) hoặc $n \leq 10$ (async). Vượt giới hạn này, chuyển sang *reachable-only* mode hoặc dùng PyBoolNet với BDD.

4.5 Một số bug phát hiện và sửa trong quá trình develop

Quá trình hiện thực bộc lộ một số bug đáng học:

1. **Bug #1 – Reserved word and/or/not:** Phiên bản đầu của parser không reject các tên biến trùng từ khoá Python. Kết quả: rule `or`, `!and` parse thành công nhưng eval sinh `SyntaxError`. Sửa bằng cách thêm danh sách `_RESERVED` kiểm tra.
2. **Bug #2 – Async không tự self-loop:** Bản đầu của `asynchronous_successors` trả về list rỗng khi state là fixed-point $\Rightarrow$ Tarjan coi state đó là sink isolated, không tạo SCC. Hậu quả: fixed-point bị bỏ sót trong async attractor. Sửa: trả về `[state]` (self-loop) khi không có diff.
3. **Bug #3 – Influence sign cho rule không monotonic:** Trường hợp `f = A xor B` (không có trong test ban đầu) làm signature ? không được trả về vì điều kiện kiểm tra `increases` and `decreases` thiếu reset. Sửa bằng thêm `return` sớm.
4. **Bug #4 – Streamlit cache vô hiệu khi đổi scheme:** Cache key đầu thiếu `scheme` $\Rightarrow$ chuyển sync $\rightarrow$ async không trigger rebuild. Sửa: bổ sung `scheme` vào key.

4.6 Tổng kết Thực nghiệm

- Phần mềm chạy đúng trên cả 5 mạng mẫu, từ 2 đến 10 node.
- Đối chiếu với PyBoolNet cho kết quả **khớp 100%**.
- 23/23 `pytest` pass.
- Hiệu năng tốt cho $n \leq 14$; vượt ngưỡng này cần đổi sang *reachable-only*.

5 Kết luận & Hướng phát triển

5.1 Tổng kết

Dề tài BOOLEAN NETWORK ANALYZER đã hoàn thành đầy đủ **toàn bộ chức năng bắt buộc** (mandatory functionality) đặt ra cho chủ đề 6 của môn *Programming Integration Project*:



1. ✓ **Parser .bnet**: Tương thích chuẩn PyBoolNet/BoolNet, xử lý đúng comment, hằng, operator dài/ngắn, từ khoá word/symbol.
2. ✓ **Influence Graph**: Suy luận dấu cạnh chính xác bằng kiểm tra monotonicity; trực quan hoá với màu xanh/đỏ/xám.
3. ✓ **State Transition Graph**: Cả sync và async, full state space hoặc reachable-only.
4. ✓ **Attractor finder**: SCC qua Tarjan, phân loại fixed-point / cyclic / complex; tính weak basin.
5. ✓ **Visualizer**: Render đẹp, đánh dấu attractor, export PNG cho báo cáo.
6. ✓ **Web UI**: Landing Page SPA trực quan hiện đại.
7. ✓ **Test & Validation**: 23 test pytest, đối chiếu PyBoolNet trên 5 mạng – khớp 100%.

Ngoài ra, phần mềm còn cung cấp các **tính năng nâng cao** (bonus):

- Mô phỏng *trajectory* từ trạng thái tự chọn.
- Hiển thị *basin size* của mỗi attractor.
- Download file `.bnet` đã được upload (tiện cho remix).
- Cảnh báo người dùng khi mạng quá lớn ($n > 12$) trước khi render.

Đối chiếu rubric:

- *Theoretical (20%)*: Mục 2 trình bày đủ định nghĩa, influence graph, sync vs async, attractor; tự diễn đạt theo Schwab *et al.* 2020.
- *Methodology & Design (25%)*: Mục 3 mô tả kiến trúc, cấu trúc dữ liệu, thuật toán cùng phức tạp.
- *Experimental & Evaluation (35%)*: Mục 4 có 5 test case + đối chiếu PyBoolNet + phân tích hiệu năng.
- *Professionalism (10%)*: src-layout, README, requirements.txt, pytest, GitHub Actions.
- *Creativity (10%)*: Async support, trajectory simulator, basin size, performance analysis.

5.2 Hạn chế

Em thẳng thắn nhìn nhận một số hạn chế:

1. **State explosion** với $n > 16$: Sinh full STG vượt thực tế. Bản hiện tại chưa hỗ trợ BDD/AIG.
2. **UI chậm khi render STG > 1024 nodes**: matplotlib layout rất chậm; trên $n = 12$ (4096 nodes) việc vẽ đã mất > 15 giây và hình không đọc được.
3. **Chưa hỗ trợ multi-valued network (MDD)**: Hiện chỉ Boolean 2 trạng thái. Các mô hình sinh học thực tế đôi khi cần 3–4 mức (e.g., thấp/vừa/cao).
4. **Chưa hỗ trợ probabilistic update**: Mọi cập nhật là deterministic (sync) hoặc non-deterministic uniform (async). Chưa có Probabilistic Boolean Network (PBN).
5. **Strong basin chưa cài đặt**: Hiện chỉ có weak basin.
6. **Chưa có cơ chế cache persistent**: Mỗi lần reload lại trang web phải sinh STG lại; nên có file cache trên đĩa.

5.3 Hướng phát triển

Các hướng mở rộng đáng theo đuổi:

1. **Tích hợp BDD/AIG backend** (qua pyeda hoặc gọi PyBoolNet): cho phép xử lý $n \geq 50$.
2. **Render STG bằng D3/Plotly thay matplotlib**: tương tác zoom/pan, click vào node để mở rộng.
3. **Multi-valued Network (MDD)**: mở rộng parser và Network engine để hỗ trợ giá trị $\{0, 1, 2, \dots, k\}$ – chuyển từ Boolean function sang Multi-valued Decision Diagram.
4. **Probabilistic Boolean Network (PBN)**: mỗi node có một danh sách rule với xác suất; phân tích Markov chain trên STG.
5. **Phân tích control kernel**: tìm tập node tối thiểu cần fix giá trị để dẫn hệ thống về attractor mong muốn (ứng dụng trong drug target identification).
6. **Generate .bnet từ tab UI**: Cho phép người dùng tạo mạng từ form chứ không cần soạn tay file.



7. **Tích hợp xuất báo cáo PDF tự động:** Sau khi phân tích, sinh ra PDF tổng hợp influence graph + STG + attractor table.
8. **Hỗ trợ collaborative editing:** nhiều người cùng edit `.bnet` trên web (qua Web-Socket).

5.4 Lời cảm ơn

Em xin chân thành cảm ơn thầy **Trịnh Văn Giang** đã hướng dẫn tận tình trong suốt quá trình thực hiện đề tài. Em cũng cảm ơn cộng đồng PyBoolNet, GINsim đã công khai mã nguồn và datasets – là cơ sở để em đối chiếu kết quả.

References

- [1] J. D. Schwab, S. D. Kühlwein, N. Ikonomi, M. Kühl, and H. A. Kestler, “Concepts in Boolean network modeling: What do they all mean?”, *Computational and Structural Biotechnology Journal*, vol. 18, pp. 571–582, 2020.
- [2] S. A. Kauffman, “Metabolic stability and epigenesis in randomly constructed genetic nets”, *Journal of Theoretical Biology*, vol. 22, no. 3, pp. 437–467, 1969.
- [3] A. Naldi, D. Berenguier, A. Fauré, F. Lopez, D. Thieffry, and C. Chaouiya, “Logical modelling of regulatory networks with GINsim 2.3”, *Biosystems*, vol. 97, no. 2, pp. 134–139, 2009.
- [4] H. Klarner, A. Bockmayr, and H. Siebert, “PyBoolNet: A Python package for the generation, analysis and visualization of Boolean networks”, *Bioinformatics*, vol. 33, no. 5, pp. 770–772, 2017.
[Online]. Available: <https://github.com/hklarner/pyboolnet>
- [5] R. Thomas, “On the relation between the logical structure of systems and their ability to generate multiple steady states or sustained oscillations”, in *Numerical Methods in the Study of Critical Phenomena*, Springer, Berlin, 1981, pp. 180–193.
- [6] T. S. Gardner, C. R. Cantor, and J. J. Collins, “Construction of a genetic toggle switch in *Escherichia coli*”, *Nature*, vol. 403, no. 6767, pp. 339–342, 2000.
- [7] M. B. Elowitz and S. Leibler, “A synthetic oscillatory network of transcriptional regulators”, *Nature*, vol. 403, no. 6767, pp. 335–338, 2000.
- [8] M. I. Davidich and S. Bornholdt, “Boolean network model predicts cell cycle sequence of fission yeast”, *PLoS ONE*, vol. 3, no. 2, p. e1672, 2008.
- [9] A. Fauré, A. Naldi, C. Chaouiya, and D. Thieffry, “Dynamical analysis of a generic Boolean model for the control of the mammalian cell cycle”, *Bioinformatics*, vol. 22, no. 14, pp. e124–e131, 2006.
- [10] R. E. Tarjan, “Depth-first search and linear graph algorithms”, *SIAM Journal on Computing*, vol. 1, no. 2, pp. 146–160, 1972.
- [11] A. A. Hagberg, D. A. Schult, and P. J. Swart, “Exploring network structure, dynamics, and function using NetworkX”, in *Proceedings of the 7th Python in Science*

- Conference (SciPy)*, Pasadena, CA, USA, 2008, pp. 11–15.
[Online]. Available: <https://networkx.org>
- [12] Streamlit Inc., “Streamlit – The fastest way to build and share data apps”, 2020.
[Online]. Available: <https://streamlit.io>
- [13] J. D. Hunter, “Matplotlib: A 2D graphics environment”, *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.
- [14] A. Wuensche, “Discrete dynamics lab: Tools for investigating cellular automata and discrete dynamical networks”, in *Artificial Life Models in Software*, Springer, 2009, pp. 215–258.
- [15] R. Albert and H. G. Othmer, “The topology of the regulatory interactions predicts the expression pattern of the segment polarity genes in *Drosophila melanogaster*”, *Journal of Theoretical Biology*, vol. 223, no. 1, pp. 1–18, 2003.